

Cours 13 — Spécifier, documenter, tester

Semaine 13 ■ Manuel Hachette ch. 8 (§ 8.1 à 8.3) ■ 2 × 1 h

Objectifs

- Écrire la spécification d'une fonction : rôle, préconditions, postcondition
- Protéger une fonction avec `assert`
- Construire un jeu de tests qui couvre les cas limites

1. Spécifier avant de coder

La **spécification** d'une fonction est son contrat :

- ce qu'elle **exige** de l'appelant : les **préconditions** (types et contraintes sur les paramètres) ;
- ce qu'elle **garantit** en retour : la **postcondition** (ce que vaut le résultat).

Le tout s'écrit dans la **docstring** :

```
def moyenne(t):  
    """Renvoie la moyenne des éléments de t.  
  
    Précondition : t est un tableau de nombres NON VIDE.  
    Postcondition : min(t) <= résultat <= max(t).  
    """
```

Si l'appelant viole la précondition (tableau vide), la fonction n'a **rien promis** : le contrat est rompu. La moitié des bugs des TP précédents étaient des préconditions implicites (« t non vide », « stock suffisant ») : maintenant on les écrit.

2. Vérifier le contrat : `assert`

```
def moyenne(t):  
    """Renvoie la moyenne de t (tableau de nombres non vide)."""  
    assert len(t) > 0, "tableau vide"  
    ...
```

`assert condition, "message"` : si la condition est fausse, le programme s'arrête avec `AssertionError` et le message. L'erreur éclate **tôt et clairement**, au lieu d'un résultat faux qui se propage.

3. Tester pour valider

3.1 Le jeu de tests

Un **jeu de tests** est une liste de cas dont on connaît la réponse. Avec `assert`, il s'exécute tout seul :

```
def test_moyenne():  
    assert moyenne([10, 20]) == 15.0  
    assert moyenne([7]) == 7.0           # cas limite : un seul  
    assert moyenne([-2, 2]) == 0.0       # négatifs  
    assert moyenne([0, 0, 0]) == 0.0     # zéros  
    print("test_moyenne : OK")
```

Un bon jeu de tests contient : des **cas normaux**, des **cas limites** (vide interdit, un seul élément, valeurs égales, extrêmes, négatifs, zéro), et pour les booléens, des cas **True et False**.

3.2 Ce que les tests peuvent (et ne peuvent pas)

Un test qui échoue **prouve** un bug ; des tests qui passent **ne prouvent pas** l'absence de bug — ils donnent confiance. D'où l'intérêt de bien choisir les cas limites : c'est là que les bugs se cachent (le `minimum(7, 7)` du TP 4!).

Comparer des **flottants** dans un test : jamais `==` ; utiliser `abs(resultat - attendu) < 1e-9` (semaine 5).

4. La méthode complète, désormais exigée

1. Écrire la **docstring** (rôle + préconditions) — *avant* le corps ;
2. écrire le **jeu de tests** — aussi avant le corps, c'est lui qui dit « fini » ;
3. écrire le corps, avec **assert** sur les préconditions ;
4. lancer les tests ; corriger jusqu'au vert.

À retenir

- Spécification = contrat : préconditions exigées, postcondition garantie, le tout dans la docstring.
- `assert condition, "message"` arrête tôt en cas de contrat violé.
- Jeu de tests = cas normaux + cas limites ; un test vert ne prouve pas, un test rouge si.
- Docstring et tests s'écrivent **avant** le corps.